COMPUTACIÓN GRÁFICA 2026-II

PASO 15: CÁMARA INTERACTIVA Y NAVEGACIÓN DE LA ESCENA

Mouse, rueda, teclado y actualización dinámica de la matriz View

Objetivo. Convertir la cámara fija del Paso 14 en una cámara orbital interactiva. El usuario podrá girar alrededor del Sistema Solar arrastrando el mouse, acercarse y alejarse con la rueda y realizar ajustes mediante teclado. La matriz View dejará de ser constante y se recalculará continuamente a partir del estado de la cámara.

1. Resultado esperado

Al finalizar este paso, el Sistema Solar podrá explorarse desde diferentes ángulos sin modificar las matrices Model de los cuerpos. El movimiento de la cámara será completamente independiente de la animación del Sol, Tierra y Luna.

Input del usuario → Estado de cámara → Posición Eye → View Matrix → Render

2. Cambio conceptual principal

Hasta el Paso 14 la cámara se definía una sola vez mediante eye, target y up. Ahora estos valores forman parte de un estado interactivo. Cada evento del usuario modifica ese estado y, en cada frame, se reconstruye la matriz View.

La escena NO se mueve para simular navegación: cambia el observador.

3. Modelo de cámara orbital

Representaremos la cámara mediante coordenadas esféricas alrededor de un objetivo (target). Usaremos tres parámetros principales: distancia, azimut y elevación.

Parámetro	Función
distancia	Radio de la cámara respecto del target. Controla zoom orbital.
azimut	Ángulo horizontal alrededor del eje Y.
elevación	Ángulo vertical. Permite observar desde arriba o abajo.
target	Punto alrededor del cual orbita la cámara.
up	Dirección superior usada por lookAt.

4. Conversión de coordenadas esféricas a cartesianas

Para construir eye calcularemos la posición de la cámara a partir de distancia, azimut y elevación:

$$x = target.x + distancia \cdot \cos(elevación) \cdot \sin(azimut)$$

$$y = target.y + distancia \cdot \sin(elevación)$$

$$z = target.z + distancia \cdot \cos(elevación) \cdot \cos(azimut)$$

5. Controles de interacción

Entrada	Acción	Variable afectada
Arrastrar mouse	Orbitar alrededor del target	azimut / elevación
Rueda del mouse	Acercar o alejar	distancia

Flechas ← →	Rotación horizontal fina	azimut
Flechas ↑ ↓	Rotación vertical fina	elevación
+ / -	Zoom fino	distancia
R	Restablecer cámara	todos los parámetros

6. Estructura del proyecto

```
sistema-solar/
│
├── index.html
├── js/
│   └── main.js
└── texturas/
    ├── sol.jpg
    └── tierra.jpg
```

La estructura física todavía no cambia. En este paso agregamos comportamiento interactivo, no nuevas clases ni nuevos archivos.

7. Código completo de index.html

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Sistema Solar 3D - Paso 15</title>

    <style>
        body {
            margin: 0;
            min-height: 100vh;
            display: flex;
            justify-content: center;
            align-items: center;
            background: #050508;
            overflow: hidden;
        }

        canvas {
            border: 1px solid #333;
            cursor: grab;
        }

        canvas.arrastrando {
            cursor: grabbing;
        }
    </style>
</head>
<body>

    <canvas id="glCanvas" width="900" height="600"></canvas>

    <script src="js/main.js"></script>
</body>
</html>
```

8. Código completo de js/main.js

```
const canvas = document.getElementById("glCanvas");
const gl = canvas.getContext("webgl2");

if (!gl) {
```

```
    throw new Error("WebGL2 no está disponible.");
}

gl.viewport(0, 0, canvas.width, canvas.height);
gl.clearColor(0.01, 0.01, 0.03, 1.0);
gl.enable(gl.DEPTH_TEST);

// ============================================================
// 1. GEOMETRÍA ESFÉRICA
// ============================================================

function crearEsfera(radio, latitudes, longitudes) {

    const vertices = [];
    const indices = [];

    for (let lat = 0; lat <= latitudes; lat++) {

        const v = lat / latitudes;
        const phi = v * Math.PI;

        for (let lon = 0; lon <= longitudes; lon++) {

            const u = lon / longitudes;
            const theta = u * Math.PI * 2.0;

            const nx = Math.sin(phi) * Math.cos(theta);
            const ny = Math.cos(phi);
            const nz = Math.sin(phi) * Math.sin(theta);

            vertices.push(
                radio * nx,
                radio * ny,
                radio * nz
            );

            vertices.push(nx, ny, nz);
            vertices.push(u, 1.0 - v);
        }
    }

    const columnas = longitudes + 1;

    for (let lat = 0; lat < latitudes; lat++) {

        for (let lon = 0; lon < longitudes; lon++) {

            const a = lat * columnas + lon;
            const b = a + columnas;

            indices.push(a, b, a + 1);
            indices.push(b, b + 1, a + 1);
        }
    }

    return {
        vertices: new Float32Array(vertices),
        indices: new Uint16Array(indices)
    };
}

const esfera = crearEsfera(1.0, 32, 48);

// ============================================================
// 2. GEOMETRÍA DE ÓRBITAS
// ============================================================

function crearOrbita(radio, segmentos = 128) {
```

```
    const puntos = [];

    for (let i = 0; i < segmentos; i++) {

        const t = i / segmentos;
        const angulo = t * Math.PI * 2.0;

        puntos.push(
            radio * Math.cos(angulo),
            0.0,
            radio * Math.sin(angulo)
        );
    }

    return new Float32Array(puntos);
}

const puntosOrbitaTierra = crearOrbita(3.5, 160);
const puntosOrbitaLuna = crearOrbita(0.95, 96);

// ============================================================
// 3. SHADERS DE CUERPOS
// ============================================================

const cuerpoVertexSource = `#version 300 es

in vec3 aPosition;
in vec3 aNormal;
in vec2 aTexCoord;

uniform mat4 uModelMatrix;
uniform mat4 uViewMatrix;
uniform mat4 uProjectionMatrix;

out vec3 vNormal;
out vec3 vWorldPosition;
out vec2 vTexCoord;

void main() {

    vec4 worldPosition =
        uModelMatrix *
        vec4(aPosition, 1.0);

    gl_Position =
        uProjectionMatrix *
        uViewMatrix *
        worldPosition;

    vWorldPosition = worldPosition.xyz;
    vNormal = normalize(mat3(uModelMatrix) * aNormal);
    vTexCoord = aTexCoord;
}
`;

const cuerpoFragmentSource = `#version 300 es

precision highp float;

in vec3 vNormal;
in vec3 vWorldPosition;
in vec2 vTexCoord;

uniform sampler2D uTexture;
uniform vec3 uLightPosition;
uniform bool uEmissive;

out vec4 outColor;
```

```
void main() {

    vec3 colorTextura =
        texture(uTexture, vTexCoord).rgb;

    if (uEmissive) {
        outColor = vec4(colorTextura, 1.0);
        return;
    }

    vec3 N = normalize(vNormal);
    vec3 L = normalize(uLightPosition - vWorldPosition);

    float difusa = max(dot(N, L), 0.0);
    float ambiente = 0.12;

    vec3 colorFinal =
        colorTextura * (ambiente + difusa);

    outColor = vec4(colorFinal, 1.0);
}
`;

// ============================================================
// 4. SHADERS DE LÍNEAS
// ============================================================

const lineaVertexSource = `#version 300 es

in vec3 aPosition;

uniform mat4 uModelMatrix;
uniform mat4 uViewMatrix;
uniform mat4 uProjectionMatrix;

void main() {

    gl_Position =
        uProjectionMatrix *
        uViewMatrix *
        uModelMatrix *
        vec4(aPosition, 1.0);
}
`;

const lineaFragmentSource = `#version 300 es

precision highp float;

uniform vec3 uColor;

out vec4 outColor;

void main() {
    outColor = vec4(uColor, 1.0);
}
`;

// ============================================================
// 5. PROGRAMAS WEBGL
// ============================================================

function crearShader(tipo, fuente) {

    const shader = gl.createShader(tipo);

    gl.shaderSource(shader, fuente);
    gl.compileShader(shader);
```

```
    if (!gl.getShaderParameter(shader, gl.COMPILE_STATUS)) {
        throw new Error(gl.getShaderInfoLog(shader));
    }

    return shader;
}

function crearPrograma(vertexSource, fragmentSource) {

    const vs = crearShader(gl.VERTEX_SHADER, vertexSource);
    const fs = crearShader(gl.FRAGMENT_SHADER, fragmentSource);

    const programa = gl.createProgram();

    gl.attachShader(programa, vs);
    gl.attachShader(programa, fs);
    gl.linkProgram(programa);

    if (!gl.getProgramParameter(programa, gl.LINK_STATUS)) {
        throw new Error(gl.getProgramInfoLog(programa));
    }

    return programa;
}

const programaCuerpo =
    crearPrograma(cuerpoVertexSource, cuerpoFragmentSource);

const programaLinea =
    crearPrograma(lineaVertexSource, lineaFragmentSource);

// ============================================================
// 6. VAO DE LA ESFERA
// ============================================================

gl.useProgram(programaCuerpo);

const vaoEsfera = gl.createVertexArray();
gl.bindVertexArray(vaoEsfera);

const vboEsfera = gl.createBuffer();
gl.bindBuffer(gl.ARRAY_BUFFER, vboEsfera);
gl.bufferData(gl.ARRAY_BUFFER, esfera.vertices, gl.STATIC_DRAW);

const eboEsfera = gl.createBuffer();
gl.bindBuffer(gl.ELEMENT_ARRAY_BUFFER, eboEsfera);
gl.bufferData(gl.ELEMENT_ARRAY_BUFFER, esfera.indices, gl.STATIC_DRAW);

const stride = 8 * Float32Array.BYTES_PER_ELEMENT;

const cuerpoPosLoc =
    gl.getAttribLocation(programaCuerpo, "aPosition");

gl.enableVertexAttribArray(cuerpoPosLoc);
gl.vertexAttribPointer(cuerpoPosLoc, 3, gl.FLOAT, false, stride, 0);

const cuerpoNormalLoc =
    gl.getAttribLocation(programaCuerpo, "aNormal");

gl.enableVertexAttribArray(cuerpoNormalLoc);
gl.vertexAttribPointer(
    cuerpoNormalLoc,
    3,
    gl.FLOAT,
    false,
    stride,
    3 * Float32Array.BYTES_PER_ELEMENT
);
```

```
const cuerpoUvLoc =
    gl.getAttribLocation(programaCuerpo, "aTexCoord");

gl.enableVertexAttribArray(cuerpoUvLoc);
gl.vertexAttribPointer(
    cuerpoUvLoc,
    2,
    gl.FLOAT,
    false,
    stride,
    6 * Float32Array.BYTES_PER_ELEMENT
);

// ============================================================
// 7. VAO DE ÓRBITAS
// ============================================================

function crearVaoOrbita(puntos) {

    gl.useProgram(programaLinea);

    const vao = gl.createVertexArray();
    gl.bindVertexArray(vao);

    const vbo = gl.createBuffer();
    gl.bindBuffer(gl.ARRAY_BUFFER, vbo);
    gl.bufferData(gl.ARRAY_BUFFER, puntos, gl.STATIC_DRAW);

    const posicionLoc =
        gl.getAttribLocation(programaLinea, "aPosition");

    gl.enableVertexAttribArray(posicionLoc);
    gl.vertexAttribPointer(
        posicionLoc,
        3,
        gl.FLOAT,
        false,
        0,
        0
    );

    return {
        vao,
        cantidad: puntos.length / 3
    };
}

const orbitaTierraGpu = crearVaoOrbita(puntosOrbitaTierra);
const orbitaLunaGpu = crearVaoOrbita(puntosOrbitaLuna);

// ============================================================
// 8. MATRICES Y VECTORES
// ============================================================

function identidad4() {
    return new Float32Array([
        1,0,0,0,
        0,1,0,0,
        0,0,1,0,
        0,0,0,1
    ]);
}

function multiplicarMat4(a, b) {

    const r = new Float32Array(16);

    for (let c = 0; c < 4; c++) {
        for (let f = 0; f < 4; f++) {
```

```
            for (let k = 0; k < 4; k++) {
                r[c * 4 + f] +=
                    a[k * 4 + f] *
                    b[c * 4 + k];
            }
        }
    }

    return r;
}

function traslacion(x, y, z) {
    return new Float32Array([
        1,0,0,0,
        0,1,0,0,
        0,0,1,0,
        x,y,z,1
    ]);
}

function escala(x, y, z) {
    return new Float32Array([
        x,0,0,0,
        0,y,0,0,
        0,0,z,0,
        0,0,0,1
    ]);
}

function rotacionY(a) {

    const c = Math.cos(a);
    const s = Math.sin(a);

    return new Float32Array([
         c,0,-s,0,
         0,1, 0,0,
         s,0, c,0,
         0,0, 0,1
    ]);
}

function rotacionZ(a) {

    const c = Math.cos(a);
    const s = Math.sin(a);

    return new Float32Array([
        c,s,0,0,
        -s,c,0,0,
        0,0,1,0,
        0,0,0,1
    ]);
}

function restar(a,b) {
    return [
        a[0]-b[0],
        a[1]-b[1],
        a[2]-b[2]
    ];
}

function normalizar(v) {

    const l = Math.hypot(v[0], v[1], v[2]);

    return [
        v[0]/l,
```

```
        v[1]/l,
        v[2]/l
    ];
}

function cruz(a,b) {
    return [
        a[1]*b[2]-a[2]*b[1],
        a[2]*b[0]-a[0]*b[2],
        a[0]*b[1]-a[1]*b[0]
    ];
}

function lookAt(eye, target, up) {

    const z = normalizar(restar(eye, target));
    const x = normalizar(cruz(up, z));
    const y = cruz(z, x);

    return new Float32Array([
        x[0],y[0],z[0],0,
        x[1],y[1],z[1],0,
        x[2],y[2],z[2],0,

        -(x[0]*eye[0] + x[1]*eye[1] + x[2]*eye[2]),
        -(y[0]*eye[0] + y[1]*eye[1] + y[2]*eye[2]),
        -(z[0]*eye[0] + z[1]*eye[1] + z[2]*eye[2]),
        1
    ]);
}

function perspectiva(fov, aspect, near, far) {

    const f = 1 / Math.tan(fov / 2);
    const nf = 1 / (near - far);

    return new Float32Array([
        f/aspect,0,0,0,
        0,f,0,0,
        0,0,(far+near)*nf,-1,
        0,0,2*far*near*nf,0
    ]);
}

// ============================================================
// 9. TEXTURAS
// ============================================================

function cargarTextura(url) {

    const textura = gl.createTexture();
    gl.bindTexture(gl.TEXTURE_2D, textura);

    gl.texImage2D(
        gl.TEXTURE_2D,
        0,
        gl.RGBA,
        1,
        1,
        0,
        gl.RGBA,
        gl.UNSIGNED_BYTE,
        new Uint8Array([255,255,255,255])
    );

    const imagen = new Image();

    imagen.onload = () => {
```

```
        gl.bindTexture(gl.TEXTURE_2D, textura);

        gl.pixelStorei(
            gl.UNPACK_FLIP_Y_WEBGL,
            true
        );

        gl.texImage2D(
            gl.TEXTURE_2D,
            0,
            gl.RGBA,
            gl.RGBA,
            gl.UNSIGNED_BYTE,
            imagen
        );

        gl.generateMipmap(gl.TEXTURE_2D);

        gl.texParameteri(
            gl.TEXTURE_2D,
            gl.TEXTURE_MIN_FILTER,
            gl.LINEAR_MIPMAP_LINEAR
        );

        gl.texParameteri(
            gl.TEXTURE_2D,
            gl.TEXTURE_MAG_FILTER,
            gl.LINEAR
        );

        gl.texParameteri(
            gl.TEXTURE_2D,
            gl.TEXTURE_WRAP_S,
            gl.REPEAT
        );

        gl.texParameteri(
            gl.TEXTURE_2D,
            gl.TEXTURE_WRAP_T,
            gl.CLAMP_TO_EDGE
        );
    };

    imagen.src = url;

    return textura;
}

function crearTexturaColor(r,g,b,a=255) {

    const textura = gl.createTexture();
    gl.bindTexture(gl.TEXTURE_2D, textura);

    gl.texImage2D(
        gl.TEXTURE_2D,
        0,
        gl.RGBA,
        1,
        1,
        0,
        gl.RGBA,
        gl.UNSIGNED_BYTE,
        new Uint8Array([r,g,b,a])
    );

    gl.texParameteri(
        gl.TEXTURE_2D,
        gl.TEXTURE_MIN_FILTER,
        gl.LINEAR
```

```
    );

    gl.texParameteri(
        gl.TEXTURE_2D,
        gl.TEXTURE_MAG_FILTER,
        gl.LINEAR
    );

    return textura;
}

const texturaSol = cargarTextura("texturas/sol.jpg");
const texturaTierra = cargarTextura("texturas/tierra.jpg");
const texturaLuna = crearTexturaColor(170,170,170);

// ============================================================
// 10. UNIFORMS
// ============================================================

const cuerpoModelLoc =
    gl.getUniformLocation(programaCuerpo, "uModelMatrix");

const cuerpoViewLoc =
    gl.getUniformLocation(programaCuerpo, "uViewMatrix");

const cuerpoProjectionLoc =
    gl.getUniformLocation(programaCuerpo, "uProjectionMatrix");

const cuerpoTextureLoc =
    gl.getUniformLocation(programaCuerpo, "uTexture");

const cuerpoLightLoc =
    gl.getUniformLocation(programaCuerpo, "uLightPosition");

const cuerpoEmissiveLoc =
    gl.getUniformLocation(programaCuerpo, "uEmissive");

const lineaModelLoc =
    gl.getUniformLocation(programaLinea, "uModelMatrix");

const lineaViewLoc =
    gl.getUniformLocation(programaLinea, "uViewMatrix");

const lineaProjectionLoc =
    gl.getUniformLocation(programaLinea, "uProjectionMatrix");

const lineaColorLoc =
    gl.getUniformLocation(programaLinea, "uColor");

// ============================================================
// 11. PROYECCIÓN
// ============================================================

const projection = perspectiva(
    60 * Math.PI / 180,
    canvas.width / canvas.height,
    0.1,
    100.0
);

// Projection no cambia en este ejemplo.
gl.useProgram(programaCuerpo);
gl.uniformMatrix4fv(
    cuerpoProjectionLoc,
    false,
    projection
);
gl.uniform1i(cuerpoTextureLoc, 0);
gl.uniform3f(cuerpoLightLoc, 0.0, 0.0, 0.0);
```

```
gl.useProgram(programaLinea);
gl.uniformMatrix4fv(
    lineaProjectionLoc,
    false,
    projection
);

// ============================================================
// 12. ESTADO DE CÁMARA INTERACTIVA
// ============================================================

const camera = {
    target: [0.0, 0.0, 0.0],

    distancia: 11.5,
    azimut: 0.0,
    elevacion: 0.42,

    distanciaMin: 3.0,
    distanciaMax: 25.0,

    sensibilidadMouse: 0.008,
    sensibilidadZoom: 0.0025
};

function limitar(valor, minimo, maximo) {
    return Math.max(
        minimo,
        Math.min(maximo, valor)
    );
}

function obtenerEyeCamara() {

    const cosElevacion =
        Math.cos(camera.elevacion);

    const x =
        camera.target[0] +
        camera.distancia *
        cosElevacion *
        Math.sin(camera.azimut);

    const y =
        camera.target[1] +
        camera.distancia *
        Math.sin(camera.elevacion);

    const z =
        camera.target[2] +
        camera.distancia *
        cosElevacion *
        Math.cos(camera.azimut);

    return [x, y, z];
}

function obtenerViewCamara() {

    const eye = obtenerEyeCamara();

    return lookAt(
        eye,
        camera.target,
        [0.0, 1.0, 0.0]
    );
}
```

```
function reiniciarCamara() {
    camera.target = [0.0, 0.0, 0.0];
    camera.distancia = 11.5;
    camera.azimut = 0.0;
    camera.elevacion = 0.42;
}

// ============================================================
// 13. INTERACCIÓN CON MOUSE
// ============================================================

let arrastrando = false;
let mouseAnteriorX = 0;
let mouseAnteriorY = 0;

canvas.addEventListener("mousedown", (evento) => {

    arrastrando = true;

    mouseAnteriorX = evento.clientX;
    mouseAnteriorY = evento.clientY;

    canvas.classList.add("arrastrando");
});

window.addEventListener("mouseup", () => {

    arrastrando = false;
    canvas.classList.remove("arrastrando");
});

window.addEventListener("mousemove", (evento) => {

    if (!arrastrando) {
        return;
    }

    const dx =
        evento.clientX -
        mouseAnteriorX;

    const dy =
        evento.clientY -
        mouseAnteriorY;

    mouseAnteriorX = evento.clientX;
    mouseAnteriorY = evento.clientY;

    camera.azimut -=
        dx *
        camera.sensibilidadMouse;

    camera.elevacion +=
        dy *
        camera.sensibilidadMouse;

    // Evitamos pasar por los polos,
    // donde el vector up puede volverse problemático.
    const limiteVertical =
        Math.PI / 2 - 0.05;

    camera.elevacion =
        limitar(
            camera.elevacion,
            -limiteVertical,
            limiteVertical
        );
});
```

```
canvas.addEventListener(
    "wheel",
    (evento) => {

        evento.preventDefault();

        camera.distancia +=
            evento.deltaY *
            camera.sensibilidadZoom;

        camera.distancia =
            limitar(
                camera.distancia,
                camera.distanciaMin,
                camera.distanciaMax
            );
    },
    { passive: false }
);

// ============================================================
// 14. INTERACCIÓN CON TECLADO
// ============================================================

window.addEventListener("keydown", (evento) => {

    const pasoAngular = 0.08;
    const pasoZoom = 0.5;

    switch (evento.key) {

        case "ArrowLeft":
            camera.azimut += pasoAngular;
            evento.preventDefault();
            break;

        case "ArrowRight":
            camera.azimut -= pasoAngular;
            evento.preventDefault();
            break;

        case "ArrowUp":
            camera.elevacion += pasoAngular;
            evento.preventDefault();
            break;

        case "ArrowDown":
            camera.elevacion -= pasoAngular;
            evento.preventDefault();
            break;

        case "+":
        case "=":
            camera.distancia -= pasoZoom;
            break;

        case "-":
            camera.distancia += pasoZoom;
            break;

        case "r":
        case "R":
            reiniciarCamara();
            break;
    }

    const limiteVertical =
        Math.PI / 2 - 0.05;
```

```
    camera.elevacion =
        limitar(
            camera.elevacion,
            -limiteVertical,
            limiteVertical
        );

    camera.distancia =
        limitar(
            camera.distancia,
            camera.distanciaMin,
            camera.distanciaMax
        );
});

// ============================================================
// 15. FUNCIONES DE DIBUJO
// ============================================================

function dibujarCuerpo(
    textura,
    modelMatrix,
    emisivo
) {
    gl.useProgram(programaCuerpo);
    gl.bindVertexArray(vaoEsfera);

    gl.activeTexture(gl.TEXTURE0);
    gl.bindTexture(gl.TEXTURE_2D, textura);

    gl.uniformMatrix4fv(
        cuerpoModelLoc,
        false,
        modelMatrix
    );

    gl.uniform1i(
        cuerpoEmissiveLoc,
        emisivo ? 1 : 0
    );

    gl.drawElements(
        gl.TRIANGLES,
        esfera.indices.length,
        gl.UNSIGNED_SHORT,
        0
    );
}

function dibujarOrbita(
    orbitaGpu,
    modelMatrix,
    color
) {
    gl.useProgram(programaLinea);
    gl.bindVertexArray(orbitaGpu.vao);

    gl.uniformMatrix4fv(
        lineaModelLoc,
        false,
        modelMatrix
    );

    gl.uniform3f(
        lineaColorLoc,
        color[0],
        color[1],
        color[2]
    );
```

```
    gl.drawArrays(
        gl.LINE_LOOP,
        0,
        orbitaGpu.cantidad
    );
}

// ============================================================
// 16. ESTADO DE ANIMACIÓN
// ============================================================

let tiempoAnterior = 0;
let giroSol = 0;
let giroTierra = 0;
let giroLuna = 0;
let orbitaTierra = 0;
let orbitaLuna = 0;

const velocidadGiroSol = 0.10;
const velocidadGiroTierra = 0.50;
const velocidadGiroLuna = 0.25;
const velocidadOrbitaTierra = 0.32;
const velocidadOrbitaLuna = 1.15;

const inclinacionLuna =
    20 * Math.PI / 180;

// ============================================================
// 17. RENDER LOOP
// ============================================================

function render(ms) {

    const tiempo = ms * 0.001;
    const dt = tiempo - tiempoAnterior;
    tiempoAnterior = tiempo;

    giroSol += velocidadGiroSol * dt;
    giroTierra += velocidadGiroTierra * dt;
    giroLuna += velocidadGiroLuna * dt;
    orbitaTierra += velocidadOrbitaTierra * dt;
    orbitaLuna += velocidadOrbitaLuna * dt;

    // ------------------------------------------------------
    // A. ACTUALIZAR VIEW DESDE EL ESTADO DE CÁMARA
    // ------------------------------------------------------

    const view = obtenerViewCamara();

    gl.useProgram(programaCuerpo);
    gl.uniformMatrix4fv(
        cuerpoViewLoc,
        false,
        view
    );

    gl.useProgram(programaLinea);
    gl.uniformMatrix4fv(
        lineaViewLoc,
        false,
        view
    );

    // ------------------------------------------------------
    // B. LIMPIAR
    // ------------------------------------------------------

    gl.clear(
```

```
    gl.COLOR_BUFFER_BIT |
    gl.DEPTH_BUFFER_BIT
);

// ---------------------------------------------------------
// C. ÓRBITA DE LA TIERRA
// ---------------------------------------------------------

dibujarOrbita(
    orbitaTierraGpu,
    identidad4(),
    [0.25, 0.35, 0.55]
);

// ---------------------------------------------------------
// D. SOL
// ---------------------------------------------------------

const modelSol =
    multiplicarMat4(
        rotacionY(giroSol),
        escala(1.35, 1.35, 1.35)
    );

dibujarCuerpo(
    texturaSol,
    modelSol,
    true
);

// ---------------------------------------------------------
// E. SISTEMA TIERRA
// ---------------------------------------------------------

const tierraSistema =
    multiplicarMat4(
        rotacionY(orbitaTierra),
        traslacion(3.5, 0.0, 0.0)
    );

const tierraVisualLocal =
    multiplicarMat4(
        rotacionY(giroTierra),
        escala(0.55, 0.55, 0.55)
    );

const modelTierra =
    multiplicarMat4(
        tierraSistema,
        tierraVisualLocal
    );

dibujarCuerpo(
    texturaTierra,
    modelTierra,
    false
);

// ---------------------------------------------------------
// F. PLANO ORBITAL DE LA LUNA
// ---------------------------------------------------------

const planoOrbitalLuna =
    multiplicarMat4(
        tierraSistema,
        rotacionZ(inclinacionLuna)
    );

dibujarOrbita(
```

```
        orbitaLunaGpu,
        planoOrbitalLuna,
        [0.55, 0.55, 0.55]
    );

    // ------------------------------------------------------
    // G. LUNA
    // ------------------------------------------------------

    const lunaOrbitaRotada =
        multiplicarMat4(
            rotacionZ(inclinacionLuna),
            rotacionY(orbitaLuna)
        );

    const lunaOrbitaLocal =
        multiplicarMat4(
            lunaOrbitaRotada,
            traslacion(0.95, 0.0, 0.0)
        );

    const lunaSistemaMundo =
        multiplicarMat4(
            tierraSistema,
            lunaOrbitaLocal
        );

    const lunaVisualLocal =
        multiplicarMat4(
            rotacionY(giroLuna),
            escala(0.18, 0.18, 0.18)
        );

    const modelLuna =
        multiplicarMat4(
            lunaSistemaMundo,
            lunaVisualLocal
        );

    dibujarCuerpo(
        texturaLuna,
        modelLuna,
        false
    );

    requestAnimationFrame(render);
}

requestAnimationFrame(render);
```

9. La matriz View ahora cambia en cada frame

En los pasos anteriores View se enviaba una sola vez porque la cámara era estática. Ahora el usuario puede modificar la cámara en cualquier momento, por lo que obtenerViewCamara() se ejecuta dentro del Render Loop y la nueva matriz se envía a ambos programas de shaders.

$$\mathbf{View(t) = lookAt(Eye(t), Target, Up)}$$

10. Separación entre estado, eventos y renderizado

Los eventos de mouse y teclado no dibujan directamente. Solamente modifican el objeto camera. El Render Loop lee ese estado y genera el resultado visual. Esta separación será muy importante cuando construyamos nuestro Mini Motor.

Evento → cambia estado | Render Loop → interpreta estado y dibuja

11. ¿Por qué limitamos la elevación?

Si la elevación alcanza exactamente ±90°, la dirección de la cámara puede alinearse con el vector up. En esa situación, el producto cruz utilizado por lookAt puede volverse degenerado. Por eso dejamos un pequeño margen antes de llegar a los polos.

12. Zoom orbital vs FOV

En este paso la rueda modifica distancia: la cámara se acerca o se aleja físicamente del target. Esto no es lo mismo que modificar FOV. Cambiar FOV altera el tipo de lente; cambiar distancia modifica la ubicación del observador.

Acción	Qué cambia
Rueda / + / -	distancia de la cámara
Modificar FOV	matriz Projection
Arrastrar mouse	azimut y elevación
Mover un planeta	matriz Model del planeta

13. Pruebas de comprensión

- Arrastrar horizontalmente el mouse y explicar por qué cambia azimut pero no la órbita de los planetas.
- Arrastrar verticalmente y observar la escena desde arriba y desde abajo.
- Usar la rueda para acercar la cámara hasta distanciaMin y explicar por qué no permitimos valores menores.
- Cambiar camera.target de [0,0,0] a [3.5,0,0] y observar qué ocurre si la Tierra no se encuentra en esa posición exacta durante toda su órbita.
- Cambiar sensibilidadMouse y comparar el comportamiento.
- Desactivar temporalmente el límite de elevación y analizar el problema cerca de ±90°.
- Cambiar FOV sin modificar distancia y comparar con el zoom por rueda.
- Eliminar la actualización de View del programaLinea y explicar por qué las órbitas dejan de coincidir visualmente con los cuerpos.
- Implementar una tecla que cambie el target entre Sol y Tierra.
- Implementar una tecla que pause únicamente el movimiento orbital sin bloquear la cámara.

14. Relación con Programación de Sistemas Interactivos

Este paso conecta directamente con el contenido oficial de Programación de Sistemas Interactivos. Ya no tenemos solamente una escena animada: existe entrada del usuario, estado, respuesta visual, navegación y un ciclo continuo de actualización.

15. Relación con nuestro Mini Motor

La lógica de cámara ya posee datos y comportamiento propios: target, distancia, ángulos, límites y cálculo de View. Esto justifica que más adelante aparezca una clase Camera o PerspectiveCamera. Del mismo modo, la separación entre eventos y renderizado prepara una arquitectura con Input/Controls y Renderer.

16. Qué NO estamos haciendo todavía

- No existe selección de planetas con mouse.
- No existe cambio automático de target hacia un cuerpo celeste.

- No implementamos paneo lateral de cámara.
- No hay interfaz gráfica con botones o información.
- No existen los demás planetas.
- No existe un fondo de estrellas.
- No hemos convertido la cámara en una clase.
- No hemos modularizado el Mini Motor.
- No utilizamos Three.js.

17. Resultado del Paso 15

Nuestro Sistema Solar ya no se limita a reproducir una animación desde un punto de vista fijo. El usuario puede explorar la escena y la matriz View se convierte en un componente dinámico del sistema gráfico.

PRIMER SISTEMA GRÁFICO 3D REALMENTE INTERACTIVO

18. Siguiente paso

Paso 16: De código monolítico a Mini Motor Gráfico. Ya acumulamos matemáticas, geometría, shaders, texturas, luces, cuerpos, líneas, cámara, interacción y jerarquías. El siguiente problema ya no será visual: será arquitectónico. Separaremos responsabilidades y comenzaremos a convertir lo construido en clases reutilizables como Vector3, Matrix4, Object3D, Geometry, Material, Mesh, Camera, Scene y Renderer, sin cambiar todavía a Three.js.